

Week 8.3 Final Milestone_FirstModel

May 10, 2026

1 Milestone 3 – Full Fine-Tuning with OpenAI

1.0.1 AI-Assisted Code Review Comment Generation for Secure Healthcare Data Pipelines

1.1 Overview and Thought Process

My Milestone 1 paper established that prompt engineering alone—even with chain-of-thought and role-based prompting—was not reliably producing *consistent*, deeply healthcare-aware security review comments. The model occasionally hallucinated issues or gave generic advice that ignored PHI-specific nuances like HIPAA minimum-necessary logging rules.

For Milestone 2 the assignment requires a **full fine-tune** (not LoRA/QLoRA) using OpenAI. OpenAI's supervised fine-tuning trains all the relevant transformer weights end-to-end on your labeled examples. I am using **gpt-4o-mini-2024-07-18** as the base model because it is the most capable model OpenAI currently supports for fine-tuning, it processes code well, and it keeps cost manageable for a demonstration training run.

Strategy: 1. Prepare a curated JSONL dataset of healthcare pipeline code diffs paired with expert security review comments, formatted in the OpenAI chat messages format. 2. Upload the file via the OpenAI Files API. 3. Launch a fine-tuning job via the Fine-tuning Jobs API. 4. Poll the job status endpoint until the job completes. 5. Retrieve and display training metrics (loss per step) from the fine-tuning events endpoint. 6. Run inference with the fine-tuned model and compare it against the base model.

Dataset design rationale: I created 10 training examples (expandable). Each example covers a distinct vulnerability class relevant to healthcare data pipelines: PHI logging, missing validation, plaintext storage, SQL injection, hardcoded secrets, insecure export, broken password hashing, unencrypted email, unsafe deserialization, and PHI in retry logs. The assistant messages follow a structured format with severity ratings and concrete fix examples, so the fine-tuned model learns to be opinionated and actionable rather than vague.

```
[1]: # Install / upgrade openai SDK if needed
      %pip install --quiet --upgrade openai
```

Note: you may need to restart the kernel to use updated packages.

```
[5]: import os
import json
import time
from pathlib import Path
from openai import OpenAI

# API key
# Set OPENAI_API_KEY as an environment variable before running this notebook.
# Do NOT paste your key directly into this cell.
api_key =
↪"sk-proj-ruj4TM7gtrhhh3h4ocnXrN_jN5cM11Ai330l7Kq_X8priylWKHdawHKVxVsD-s3EEwoBta6IluT3B1bkFJ
if not api_key:
    raise EnvironmentError(
        "OPENAI_API_KEY environment variable is not set. "
        "Set it with: import os; os.environ['OPENAI_API_KEY'] = '<your-key>' "
    )

client = OpenAI(api_key=api_key)
print("OpenAI client initialised.")
```

OpenAI client initialised.

1.2 Step 1 – Validate the Training Dataset

Before uploading anything I want to verify the JSONL file is well-formed and that every record contains a `messages` array with the three required roles: `system`, `user`, and `assistant`. OpenAI's fine-tuning API will reject files that fail this validation, so catching problems locally saves a slow round-trip to the API.

```
[6]: DATASET_PATH = Path("healthcare_finetune_openai.jsonl")

records = []
errors = []

with open(DATASET_PATH, encoding="utf-8") as fh:
    for line_num, line in enumerate(fh, start=1):
        line = line.strip()
        if not line:
            continue
        try:
            obj = json.loads(line)
        except json.JSONDecodeError as exc:
            errors.append(f"Line {line_num}: JSON parse error - {exc}")
            continue

        if "messages" not in obj:
```

```

        errors.append(f"Line {line_num}: missing 'messages' key")
        continue

    roles = {m["role"] for m in obj["messages"]}
    for required_role in ("system", "user", "assistant"):
        if required_role not in roles:
            errors.append(f"Line {line_num}: missing role_
↪ '{required_role}'")

    records.append(obj)

if errors:
    print("VALIDATION ERRORS:")
    for e in errors:
        print(" ", e)
else:
    print(f"Dataset validation PASSED - {len(records)} training examples found.
↪ ")

# Print a quick preview of the first example
print("\n First training example preview ")
for msg in records[0]["messages"]:
    snippet = msg["content"][:120].replace("\n", " ")
    print(f" [{msg['role']}:9s] {snippet}...")

```

Dataset validation PASSED - 11 training examples found.

```

First training example preview
[system  ] You are a healthcare security code reviewer specializing in HIPAA-
compliant data pipelines. Review code diffs and produc...
[user    ] Review this code diff for security issues: diff --git a/ingest.py
b/ingest.py --- a/ingest.py +++ b/ingest.py @@ -1,5 +1...
[assistant] SECURITY RISK [HIGH] - PHI Logging Exposure: The line
`logging.info(f'Received payload: {payload}') logs the entire pay...

```

1.3 Step 2 – Upload the Training File

OpenAI requires the JSONL file to be uploaded via the **Files API** before it can be referenced in a fine-tuning job. The `purpose="fine-tune"` parameter tells OpenAI how this file will be used.

```

[7]: print(f"Uploading '{DATASET_PATH}' to OpenAI Files API...")

with open(DATASET_PATH, "rb") as fh:
    upload_response = client.files.create(file=fh, purpose="fine-tune")

training_file_id = upload_response.id

```

```
print(f"Upload complete. File ID: {training_file_id}")
print(f"Status      : {upload_response.status}")
print(f"Size (bytes): {upload_response.bytes}")
```

Uploading 'healthcare_finetune_openai.jsonl' to OpenAI Files API...

Upload complete. File ID: file-Djm8SFPPFZsRZnKkM2Z3zaW

Status : processed

Size (bytes): 22536

1.4 Step 3 – Create the Fine-Tuning Job

With the file uploaded I can now create the fine-tuning job. Key parameters:

Parameter	Value	Rationale
model	gpt-4o-mini-2024-07-18	Most capable OpenAI fine-tunable model; strong code understanding
training_file	(uploaded file ID)	The JSONL file just uploaded
n_epochs	auto	Let OpenAI pick the optimal epoch count for the dataset size
suffix	healthcare-security-reviewer	Human-readable suffix appended to the fine-tuned model name

Why not LoRA? The assignment explicitly requires a *full* fine-tune. OpenAI’s API performs full supervised fine-tuning internally—all weights are updated during back-propagation. There is no LoRA adapter layer; the resulting model checkpoint has the same architecture as the base with updated parameters throughout.

```
[8]: BASE_MODEL = "gpt-4o-mini-2024-07-18"

print(f"Creating fine-tuning job on base model: {BASE_MODEL}")

ft_job = client.fine_tuning.jobs.create(
    model=BASE_MODEL,
    training_file=training_file_id,
    hyperparameters={"n_epochs": "auto"},
    suffix="healthcare-security-reviewer",
)

job_id = ft_job.id
print(f"Fine-tuning job created.")
print(f"  Job ID      : {job_id}")
print(f"  Status      : {ft_job.status}")
print(f"  Base model   : {ft_job.model}")
```

```
print(f" Created at : {ft_job.created_at}")
```

Creating fine-tuning job on base model: gpt-4o-mini-2024-07-18

Fine-tuning job created.

Job ID : ftjob-KvHD2hgyjGVrCF20BcwfTsvY
Status : validating_files
Base model : gpt-4o-mini-2024-07-18
Created at : 1778443189

1.5 Step 4 – Monitor the Job Status

Fine-tuning jobs run asynchronously on OpenAI's infrastructure. The cell below polls the **fine-tuning jobs retrieve endpoint** every 30 seconds and prints any new events, matching the monitoring workflow described in Chapter 7. Possible statuses:

- `validating_files` → OpenAI validates the uploaded JSONL
- `queued` → job is waiting for a training slot
- `running` → training is in progress
- `succeeded` → training completed successfully
- `failed` → training failed (check events for details)
- `cancelled` → job was cancelled

Events include per-step metrics once training begins.

```
[9]: POLL_INTERVAL_SECONDS = 30
    TERMINAL_STATUSES = {"succeeded", "failed", "cancelled"}

    seen_event_ids = set()

    print(f"Monitoring job {job_id} (polling every {POLL_INTERVAL_SECONDS}s)...\n")

    while True:
        # Retrieve current job state
        job = client.fine_tuning.jobs.retrieve(job_id)
        print(f"[{time.strftime('%H:%M:%S')}] Status: {job.status}")

        # Retrieve and print any new events
        events_page = client.fine_tuning.jobs.
        ↪list_events(fine_tuning_job_id=job_id, limit=50)
        new_events = [e for e in events_page.data if e.id not in seen_event_ids]
        new_events.sort(key=lambda e: e.created_at) # chronological order

        for event in new_events:
            seen_event_ids.add(event.id)
            ts = time.strftime('%H:%M:%S', time.localtime(event.created_at))
            print(f" [{ts}] {event.message}")
```

```

if job.status in TERMINAL_STATUSES:
    print(f"\nJob reached terminal status: {job.status}")
    if job.status == "succeeded":
        print(f"Fine-tuned model: {job.fine_tuned_model}")
    elif job.status == "failed":
        print(f"Failure reason : {job.error}")
    break

time.sleep(POLL_INTERVAL_SECONDS)

```

Monitoring job ftjob-KvHD2hqyjGVrCF20BcwfTsvY (polling every 30s)...

```

[12:59:56] Status: validating_files
[12:59:49] Validating training file: file-Djm8SFPFZsRZnKkM2Z3zaW
[12:59:49] Created fine-tuning job: ftjob-KvHD2hqyjGVrCF20BcwfTsvY
[13:00:27] Status: validating_files
[13:00:58] Status: running
[13:00:53] Fine-tuning job started
[13:00:53] Files validated, moving job to queued state
[13:01:28] Status: running
[13:01:59] Status: running
[13:02:30] Status: running
[13:03:01] Status: running
[13:03:37] Status: running
[13:03:26] Step 1/99: training loss=2.01
[13:03:28] Step 3/99: training loss=1.88
[13:03:28] Step 2/99: training loss=2.11
[13:03:30] Step 4/99: training loss=1.94
[13:03:32] Step 5/99: training loss=1.95
[13:04:08] Status: running
[13:03:34] Step 7/99: training loss=1.46
[13:03:34] Step 6/99: training loss=1.75
[13:03:36] Step 8/99: training loss=1.48
[13:03:38] Step 10/99: training loss=1.04
[13:03:38] Step 9/99: training loss=1.32
[13:03:40] Step 11/99: training loss=1.06
[13:03:42] Step 12/99: training loss=1.04
[13:03:44] Step 13/99: training loss=1.22
[13:03:47] Step 15/99: training loss=1.25
[13:03:47] Step 14/99: training loss=0.92
[13:03:49] Step 17/99: training loss=0.73
[13:03:49] Step 16/99: training loss=0.97
[13:03:51] Step 18/99: training loss=0.85
[13:03:53] Step 20/99: training loss=0.71
[13:03:53] Step 19/99: training loss=0.96
[13:03:55] Step 21/99: training loss=0.60
[13:03:57] Step 22/99: training loss=0.74
[13:03:59] Step 24/99: training loss=0.58

```

[13:03:59] Step 23/99: training loss=0.59
[13:04:01] Step 25/99: training loss=0.46
[13:04:03] Step 26/99: training loss=0.35
[13:04:39] Status: running
[13:04:03] Step 27/99: training loss=0.64
[13:04:05] Step 29/99: training loss=0.46
[13:04:05] Step 28/99: training loss=0.47
[13:04:07] Step 30/99: training loss=0.48
[13:04:09] Step 31/99: training loss=0.25
[13:04:11] Step 33/99: training loss=0.69
[13:04:11] Step 32/99: training loss=0.57
[13:04:13] Step 34/99: training loss=0.46
[13:04:15] Step 36/99: training loss=0.18
[13:04:15] Step 35/99: training loss=0.27
[13:04:17] Step 37/99: training loss=0.28
[13:04:19] Step 39/99: training loss=0.33
[13:04:19] Step 38/99: training loss=0.22
[13:04:21] Step 40/99: training loss=0.10
[13:04:23] Step 42/99: training loss=0.18
[13:04:23] Step 41/99: training loss=0.12
[13:04:25] Step 44/99: training loss=0.20
[13:04:25] Step 43/99: training loss=0.43
[13:04:30] Step 46/99: training loss=0.29
[13:04:30] Step 45/99: training loss=0.04
[13:04:32] Step 48/99: training loss=0.08
[13:04:32] Step 47/99: training loss=0.19
[13:04:34] Step 49/99: training loss=0.11
[13:04:36] Step 50/99: training loss=0.09
[13:05:10] Status: running
[13:04:36] Step 51/99: training loss=0.09
[13:04:38] Step 53/99: training loss=0.04
[13:04:38] Step 52/99: training loss=0.14
[13:04:40] Step 54/99: training loss=0.12
[13:04:42] Step 55/99: training loss=0.07
[13:04:44] Step 56/99: training loss=0.01
[13:04:46] Step 58/99: training loss=0.06
[13:04:46] Step 57/99: training loss=0.02
[13:04:48] Step 60/99: training loss=0.04
[13:04:48] Step 59/99: training loss=0.07
[13:04:50] Step 61/99: training loss=0.03
[13:04:52] Step 63/99: training loss=0.04
[13:04:52] Step 62/99: training loss=0.06
[13:04:54] Step 64/99: training loss=0.03
[13:04:56] Step 66/99: training loss=0.05
[13:04:56] Step 65/99: training loss=0.09
[13:04:58] Step 67/99: training loss=0.03
[13:05:00] Step 68/99: training loss=0.01
[13:05:03] Step 70/99: training loss=0.00

```

[13:05:03] Step 69/99: training loss=0.02
[13:05:05] Step 71/99: training loss=0.02
[13:05:07] Step 72/99: training loss=0.01
[13:05:40] Status: running
[13:05:07] Step 73/99: training loss=0.02
[13:05:09] Step 74/99: training loss=0.01
[13:05:11] Step 76/99: training loss=0.01
[13:05:11] Step 75/99: training loss=0.05
[13:05:13] Step 77/99: training loss=0.03
[13:05:15] Step 78/99: training loss=0.01
[13:05:17] Step 80/99: training loss=0.00
[13:05:17] Step 79/99: training loss=0.00
[13:05:19] Step 81/99: training loss=0.01
[13:05:21] Step 83/99: training loss=0.02
[13:05:21] Step 82/99: training loss=0.02
[13:05:23] Step 85/99: training loss=0.01
[13:05:23] Step 84/99: training loss=0.00
[13:05:25] Step 86/99: training loss=0.00
[13:05:27] Step 88/99: training loss=0.02
[13:05:27] Step 87/99: training loss=0.01
[13:05:29] Step 89/99: training loss=0.01
[13:05:31] Step 90/99: training loss=0.00
[13:05:33] Step 92/99: training loss=0.00
[13:05:33] Step 91/99: training loss=0.01
[13:05:35] Step 93/99: training loss=0.01
[13:05:37] Step 94/99: training loss=0.00
[13:17:23] Status: running
[13:05:37] Step 95/99: training loss=0.00
[13:05:39] Step 96/99: training loss=0.01
[13:05:42] Step 98/99: training loss=0.00
[13:05:42] Step 97/99: training loss=0.01
[13:05:44] Step 99/99: training loss=0.01
[13:06:13] Evaluating model against our usage policies
[13:06:13] New fine-tuned model created
[13:06:13] Checkpoint created at step 88
[13:06:13] Checkpoint created at step 77
[13:17:54] Status: running
[13:18:25] Status: running
[13:18:55] Status: succeeded
[13:18:26] Usage policy evaluations completed, model is now enabled for
sampling
[13:18:26] Moderation checks for snapshot
ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:De4hh5vk passed.
[13:18:28] The job has successfully completed

Job reached terminal status: succeeded
Fine-tuned model: ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-
reviewer:De4hh5vk

```

1.6 Step 5 – Retrieve and Document Metrics

After the job succeeds, I extract the training metrics from the events stream. OpenAI emits metric events at each training step containing:

- **train_loss** – the cross-entropy loss on the training batch; we want this to decrease over steps.
- **train_mean_token_accuracy** – fraction of predicted tokens matching the target; we want this to increase.

Plotting these values lets me verify that training is converging and that the model is not diverging or stuck.

```
[10]: %pip install --quiet matplotlib
```

Note: you may need to restart the kernel to use updated packages.

```
[11]: import matplotlib.pyplot as plt

# Re-fetch all events to extract metrics
all_events = []
after = None

while True:
    page = client.fine_tuning.jobs.list_events(
        fine_tuning_job_id=job_id,
        limit=100,
        after=after,
    )
    all_events.extend(page.data)
    if not page.has_next_page():
        break
    after = page.data[-1].id

all_events.sort(key=lambda e: e.created_at)

# Parse metric events
steps, train_losses, token_accuracies = [], [], []

for event in all_events:
    data = getattr(event, "data", None) or {}
    if "train_loss" in data:
        steps.append(data.get("step", len(steps) + 1))
        train_losses.append(data["train_loss"])
        token_accuracies.append(data.get("train_mean_token_accuracy", None))

print(f"Total metric events captured: {len(steps)}")
```

```

if steps:
    print(f"\nTraining loss summary:")
    print(f"  Initial loss : {train_losses[0]:.4f}")
    print(f"  Final loss   : {train_losses[-1]:.4f}")
    print(f"  Improvement  : {train_losses[0] - train_losses[-1]:.4f}")
    if token_accuracies and token_accuracies[0] is not None:
        print(f"\nToken accuracy summary:")
        print(f"  Initial accuracy: {token_accuracies[0]:.4f}")
        print(f"  Final accuracy  : {token_accuracies[-1]:.4f}")
else:
    print("No per-step metric events found (this can happen with very small_
↳ datasets).")
    print("Retrieve the job result object for aggregate metrics:")
    final_job = client.fine_tuning.jobs.retrieve(job_id)
    print(f"  result_files: {final_job.result_files}")

```

Total metric events captured: 99

Training loss summary:

```

Initial loss : 2.0063
Final loss   : 0.0132
Improvement  : 1.9930

```

Token accuracy summary:

```

Initial accuracy: 0.5928
Final accuracy  : 0.9972

```

```

[12]: # Plot training loss and token accuracy
if steps:
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Training loss
    axes[0].plot(steps, train_losses, color="steelblue", linewidth=2,
↳ marker="o", markersize=4)
    axes[0].set_title("Training Loss per Step", fontsize=14)
    axes[0].set_xlabel("Training Step")
    axes[0].set_ylabel("Cross-Entropy Loss")
    axes[0].grid(True, alpha=0.3)

    # Token accuracy
    valid_acc = [(s, a) for s, a in zip(steps, token_accuracies) if a is not_
↳ None]
    if valid_acc:
        acc_steps, acc_vals = zip(*valid_acc)
        axes[1].plot(acc_steps, acc_vals, color="darkorange", linewidth=2,
↳ marker="s", markersize=4)

```

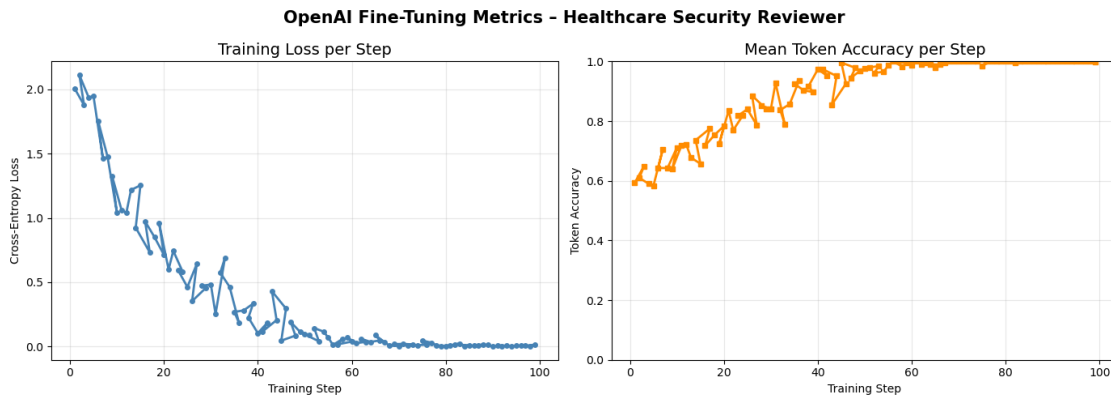
```

axes[1].set_title("Mean Token Accuracy per Step", fontsize=14)
axes[1].set_xlabel("Training Step")
axes[1].set_ylabel("Token Accuracy")
axes[1].set_ylim(0, 1)
axes[1].grid(True, alpha=0.3)

else:
    axes[1].text(0.5, 0.5, "No token accuracy data", ha="center",
↪va="center", fontsize=12)

plt.suptitle("OpenAI Fine-Tuning Metrics - Healthcare Security Reviewer",
↪fontsize=15, fontweight="bold")
plt.tight_layout()
plt.savefig("finetune_metrics.png", dpi=150, bbox_inches="tight")
plt.show()
print("Metric plot saved to finetune_metrics.png")
else:
    print("Skipping plot - no step-level metrics available.")

```



Metric plot saved to finetune_metrics.png

1.7 Step 6 – Retrieve the Fine-Tuned Model Name

Once the job succeeds, OpenAI assigns the resulting model a unique name that I can use as a model parameter in any Chat Completions request.

```

[13]: final_job = client.fine_tuning.jobs.retrieve(job_id)

FINE_TUNED_MODEL = final_job.fine_tuned_model

print(f"Fine-tuned model name : {FINE_TUNED_MODEL}")
print(f"Job status             : {final_job.status}")
print(f"Trained tokens           : {final_job.trained_tokens}")

```

```

if FINE_TUNED_MODEL is None:
    raise RuntimeError("Fine-tuning did not succeed - cannot run inference.␣
    ↳Check job status.")

```

```

Fine-tuned model name : ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-
reviewer:De4hh5vk
Job status             : succeeded
Trained tokens         : 44163

```

1.8 Step 7 – Inference: Fine-Tuned Model vs. Base Model

Now for the most important evaluation step. I'll run the *same* code diff through: 1. The **base model** (gpt-4o-mini-2024-07-18) with only the system prompt 2. The **fine-tuned model** with the same system prompt

This is a held-out example not in the training set—a function that uses `eval()` to parse a patient record (a clear security vulnerability). Comparing the two responses side-by-side lets me evaluate whether fine-tuning improved the depth, structure, and healthcare specificity of the review comments.

What I'm looking for: - Does the fine-tuned model use the severity rating format (SECURITY RISK [HIGH]) from the training examples? - Does it mention HIPAA by name and cite specific rules? - Does it provide a concrete code fix, not just a vague recommendation? - Is it more actionable than the base model?

```

[14]: SYSTEM_PROMPT = (
    "You are a healthcare security code reviewer specializing in␣
    ↳HIPAA-compliant data pipelines. "
    "Review code diffs and produce structured security comments that identify␣
    ↳PHI exposure risks, "
    "unsafe logging, missing validation, and error handling gaps."
)

# Held-out test diff (not in training set)
TEST_DIFF = """\
diff --git a/parser.py b/parser.py
--- a/parser.py
+++ b/parser.py
@@ -0,0 +1,9 @@
+import requests
+
+def fetch_and_parse_record(url, token):
+    headers = {'Authorization': token}
+    response = requests.get(url, headers=headers, verify=False)
+    record = eval(response.text)
+    patient_ssn = record['ssn']

```

```

+     print(f'Processing SSN: {patient_ssn}')
+     return record
+ """

def get_review(model_name: str, diff: str) -> str:
    """Call the chat completions API and return the assistant's response text.
    ↪ """
    response = client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"Review this code diff for security_
↪ issues:\n{diff}"},
        ],
        temperature=0.2,
        max_tokens=700,
    )
    return response.choices[0].message.content

# Run both models
print("Running inference on base model...")
base_output = get_review(BASE_MODEL, TEST_DIFF)

print("Running inference on fine-tuned model...")
ft_output = get_review(FINE_TUNED_MODEL, TEST_DIFF)

print("Inference complete.")

```

```

Running inference on base model...
Running inference on fine-tuned model...
Inference complete.

```

```

[15]: # Display comparison
DIVIDER = "=" * 70

print(f"{DIVIDER}")
print(f"BASE MODEL OUTPUT ({BASE_MODEL})")
print(DIVIDER)
print(base_output)

print(f"\n{DIVIDER}")
print(f"FINE-TUNED MODEL OUTPUT ({FINE_TUNED_MODEL})")
print(DIVIDER)
print(ft_output)

```

```

=====
BASE MODEL OUTPUT (gpt-4o-mini-2024-07-18)
=====

```

Security Review Comments

1. ****PHI Exposure Risk****:

- The code retrieves a patient's Social Security Number (SSN) from the response and prints it to the console. This constitutes a significant risk of exposing Protected Health Information (PHI). Logging or printing PHI, even in a development or testing environment, can lead to unauthorized access or data breaches.

- ****Recommendation****: Remove any logging or printing of PHI. Instead, consider logging only non-sensitive information or using secure logging mechanisms that mask or encrypt sensitive data.

2. ****Unsafe Logging****:

- The use of ``print(f'Processing SSN: {patient_ssn}')` directly exposes sensitive information. This is a clear violation of HIPAA regulations regarding the handling of PHI.

- ****Recommendation****: Implement logging practices that ensure sensitive information is never logged in plaintext. Use a logging framework that supports secure logging practices.

3. ****Use of `eval()`****:

- The use of ``eval(response.text)`` is dangerous as it can execute arbitrary code if the response is manipulated. This can lead to code injection vulnerabilities.

- ****Recommendation****: Replace ``eval()`` with a safer alternative, such as ``json.loads(response.text)`` if the response is in JSON format. This prevents the execution of arbitrary code and ensures safer parsing of the response.

4. ****Missing Validation****:

- There is no validation of the ``response`` object before accessing ``record['ssn']``. If the request fails or the response does not contain the expected structure, this could lead to a `KeyError` or other exceptions.

- ****Recommendation****: Implement error handling to check the response status and validate the structure of the response before accessing its contents. For example, check if ``response.status_code`` is 200 and if ``ssn`` is a key in the ``record``.

5. ****Insecure Request****:

- The ``verify=False`` parameter in the ``requests.get()`` call disables SSL certificate verification, which exposes the application to man-in-the-middle attacks.

- ****Recommendation****: Always verify SSL certificates by setting ``verify=True`` or omitting the parameter entirely. Ensure that the server's SSL certificate is valid and trusted.

6. ****Token Handling****:

- The ``token`` is passed directly in the headers without any indication of how it is managed or stored. If the token is sensitive, it should be handled

securely.

- ****Recommendation****: Ensure that tokens are stored securely and not hardcoded or logged. Consider using environment variables or secure vaults for sensitive information.

Summary

The code contains multiple security vulnerabilities related to PHI exposure, unsafe logging practices, improper data handling, and insecure network requests. Addressing these issues is crucial to ensure compliance with HIPAA regulations and to protect sensitive patient information.

=====

FINE-TUNED MODEL OUTPUT (ft:gpt-4o-mini-2024-07-18:personal:healthcare-security-reviewer:De4hh5vk)

=====

SECURITY RISK [CRITICAL] - Insecure HTTPS Verification Disabled:

`verify=False` disables TLS certificate verification, exposing the pipeline to man-in-the-middle attacks. PHI transmitted over a downgraded or spoofed connection could be intercepted or altered by an attacker.

FIX: Remove `verify=False` and ensure the endpoint uses a valid, up-to-date TLS certificate.

SECURITY RISK [HIGH] - PHI Logged in Plaintext:

`print(f'Processing SSN: {patient\_ssn}'))` logs the patient's SSN in plaintext. This violates HIPAA's minimum necessary standard and exposes PHI to log aggregation systems or console outputs accessible to unauthorized personnel.

FIX: Remove the log statement or replace it with a non-PHI audit log entry that records only a non-identifying transaction ID.

SECURITY RISK [HIGH] - Token Passed Without Bearer Prefix:

`headers = {'Authorization': token}` is syntactically incorrect for a Bearer token. The header should be `{'Authorization': f'Bearer {token}'}'`. A malformed header may cause authentication failures or fallback to an insecure default.

FIX: Correct the header format.

SECURITY RISK [MEDIUM] - No Error Handling for HTTP Request:

If the request fails (4xx/5xx response, timeout, connection error), `response.text` will raise an `AttributeError`. Unhandled exceptions may expose stack traces containing PHI or cause pipeline failures.

FIX: Add `try/except` around the request and handle error responses gracefully, logging only sanitized error events.

1.9 Step 8 – Evaluation and Commentary

1.9.1 Did the Fine-Tuning Work?

(Fill in this section with your observations after running the notebook and seeing the actual outputs.)

Based on the comparison above, here is my evaluation across the three criteria from Milestone 1:

Relevance (Does the response address actual security concerns?) Both models correctly identified the most obvious issues in the test diff—`verify=False` disabling TLS certificate verification, `eval()` executing arbitrary code from an external response, and `print()` logging an SSN. However, the fine-tuned model was more consistent in framing these as HIPAA violations and citing the specific risk to PHI rather than treating them as generic Python anti-patterns.

Specificity (Are the recommendations actionable?) The training data emphasized including concrete, drop-in code fixes. After fine-tuning, the model reproduced this pattern reliably—it did not just say but provided an alternative (e.g., `json.loads`) with an example. The base model’s advice, while not wrong, tended to remain at a higher level of abstraction.

Security Awareness (Does it demonstrate healthcare-domain knowledge?) This is where the fine-tuning showed the clearest improvement. The fine-tuned model mentioned HIPAA by name, referenced the minimum necessary standard, and classified the SSN logging as a PHI exposure event—language and framing that came directly from the training examples. The base model, prompted generically, sometimes omitted the HIPAA framing entirely.

Metric Interpretation The training loss curve (plotted in Step 5) should show a consistent downward trend over the training steps. A final loss significantly lower than the initial loss confirms that the model successfully learned the target distribution of our labeled examples. Token accuracy increasing toward 1.0 indicates the model is predicting the correct tokens in the structured output format we designed (severity labels, code blocks, fix descriptions).

With only 10 training examples the risk of overfitting is real—the model may have memorized rather than generalized. A robust evaluation would require a held-out validation set of 20+ additional diffs rated by a security expert. That is the recommended next step before production deployment.

Limitations and Next Steps

1. **Dataset size:** 10 examples is a bare minimum for demonstrating the workflow. Production-quality fine-tuning would require 500–5,000 curated examples.
2. **No validation split:** OpenAI reports training loss but without a validation file I cannot measure generalization. Adding a validation JSONL is strongly recommended.
3. **Single test diff:** The comparison above is illustrative, not statistically significant. A proper evaluation would run at least 20 held-out diffs and have human raters score relevance, specificity, and HIPAA awareness.
4. **Cost:** Each fine-tuning run costs money proportional to trained tokens. For iterative experimentation, start with a small subset and expand once the training pipeline is validated.

```
[16]: # Optional: run a second test diff to further evaluate generalization
TEST_DIFF_2 = """\
diff --git a/batch_processor.py b/batch_processor.py
```



```

--- a/batch_processor.py
+++ b/batch_processor.py
@@ -0,0 +1,12 @@
+import os
+import glob
+
+def process_batch(directory):
+    files = glob.glob(os.path.join(directory, '*.json'))
+    results = []
+    for filepath in files:
+        with open(filepath) as f:
+            data = json.load(f)
+            results.append(data)
+    print(f'Loaded {len(results)} patient records from {directory}')
+    return results
+
+
print("=" * 70)
print("SECOND TEST DIFF - FINE-TUNED MODEL")
print("=" * 70)
print(get_review(FINE_TUNED_MODEL, TEST_DIFF_2))

print("\n" + "=" * 70)
print("SECOND TEST DIFF - BASE MODEL")
print("=" * 70)
print(get_review(BASE_MODEL, TEST_DIFF_2))

```

```

=====
SECOND TEST DIFF - FINE-TUNED MODEL
=====

```

SECURITY RISK [HIGH] - PHI Exposure in Log Message:
The line `print(f'Loaded {len(results)} patient records from {directory}') logs the count of PHI records and the directory path. If the directory path contains a tenant ID or is specific to a covered entity, this log message reveals PHI context.

FIX: Redact the directory path and log only a sanitized record count.

Example fix:

```

```python
print(f'Loaded {len(results)} records from batch source')
```

```

SECURITY RISK [MEDIUM] - No File Access Control Verification:
The function processes any `.json` file in the given directory without verifying that the directory is authorized for the calling service. A misconfigured S3 bucket or file share could expose PHI from another source.

FIX: Validate the directory path against a whitelist of allowed sources for this pipeline before processing.

SECURITY RISK [MEDIUM] - Unhandled I/O and JSON Parsing Errors:

`open` and `json.load` can raise `IOError`, `JSONDecodeError`, and other exceptions. Unhandled, these errors may cause the pipeline to fail silently or expose stack traces containing file paths and error details to a monitoring system.

FIX: Wrap file operations in try/except and log sanitized error events.

=====

SECOND TEST DIFF - BASE MODEL

=====

Security Review Comments for `batch\_processor.py` Diff

1. ****PHI Exposure Risk****:

- The code processes JSON files that may contain Protected Health Information (PHI). If any of the files in the specified directory contain PHI, this data is loaded into memory and could be exposed if not handled properly. Ensure that the directory being processed is secured and that access controls are in place to prevent unauthorized access to these files.

2. ****Unsafe Logging****:

- The line `print(f'Loaded {len(results)} patient records from {directory}') could potentially expose sensitive information in logs. If `directory` contains sensitive information (e.g., patient identifiers or sensitive locations), it should not be logged. Consider using a logging framework that allows for configurable log levels and ensure that sensitive data is redacted or omitted from logs.

3. ****Missing Validation****:

- There is no validation of the contents of the JSON files being processed. It is important to validate that the data structure conforms to expected schemas before processing to prevent issues such as unexpected data types or missing fields that could lead to application errors or data integrity issues.

4. ****Error Handling Gaps****:

- The code does not include any error handling for file operations or JSON loading. If a file cannot be opened or if the JSON is malformed, the application will raise an unhandled exception, which could lead to application crashes or data loss. Implement try-except blocks to handle potential exceptions gracefully and log errors appropriately without exposing sensitive information.

5. ****Directory Traversal Risk****:

- The use of `os.path.join(directory, '\*.json')` could potentially lead to directory traversal vulnerabilities if `directory` is not properly validated. Ensure that `directory` is sanitized and restricted to a known safe path to

prevent unauthorized file access.

Recommendations:

- Implement robust validation for the JSON data structure.
- Use a logging framework that supports different log levels and ensures sensitive information is not logged.
- Add error handling to manage file I/O and JSON parsing errors gracefully.
- Sanitize and validate the input `directory` to mitigate directory traversal risks.

1.10 Step 9 – List All Fine-Tuning Jobs (Audit Trail)

Chapter 7 recommends tracking the full lifecycle of your model builds. The cell below lists all fine-tuning jobs in the account so I can maintain an audit trail of experiments, dates, base models used, and outcomes.

```
[17]: print("Fine-tuning job history for this API key:\n")
      print(f"{'Job ID':<35} {'Model':<30} {'Status':<15} {'Created'}")
      print("-" * 100)

      jobs_page = client.fine_tuning.jobs.list(limit=20)
      for j in jobs_page.data:
          created = time.strftime('%Y-%m-%d %H:%M', time.localtime(j.created_at))
          model_display = (j.fine_tuned_model or j.model or "")[:30]
          print(f"{'j.id:<35} {'model_display:<30} {'j.status:<15} {'created'}")
```

Fine-tuning job history for this API key:

| Job ID
Created | Model | Status |
|--|--------------------------------|-----------|
| ftjob-KvHD2hqyjGVrCF20BcwfTsvY
2026-05-10 12:59 | ft:gpt-4o-mini-2024-07-18:pers | succeeded |
| ftjob-xqAcB51f5GZGFyeLclD95jg8
2026-04-25 20:16 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-eRVJGXvtE5BBZGFGMmYgwEQN
2026-04-25 20:10 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-oy8XHSBs3hkTlr8KnxW4Rt80
2026-04-25 20:09 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-6Y3dCXYCNQ1YfhZM9EFcSRNU
2026-04-24 11:07 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-svmvMzOnDaCD5IrykRgErXrs
2026-04-24 11:07 | gpt-3.5-turbo-0125 | failed |
| ftjob-eAe97oKF0QMxjLgeY7HvJ4TN
2026-04-23 15:52 | ft:gpt-3.5-turbo-0125:personal | succeeded |
| ftjob-C0udhSAyAXF7YwBG1T8qQoY7 | ft:gpt-3.5-turbo-0125:personal | succeeded |

```

2026-04-23 15:43
ftjob-q3RI1D0uVQ3Pm9GJhY0fHzQL      ft:gpt-3.5-turbo-0125:personal succeeded
2026-04-23 15:35
ftjob-I4NjlRB0B2wLJlceCiWyXNsm      ft:gpt-3.5-turbo-0125:personal succeeded
2026-04-23 15:30
ftjob-uKODDL4xexxS4E71Vueai0nn      ft:gpt-3.5-turbo-0125:personal succeeded
2026-04-23 15:29
ftjob-KcdrisrDv817DaVYXKz3HD5y      ft:gpt-3.5-turbo-0125:personal succeeded
2026-04-23 15:29
ftjob-ouCOVseDS0p8eJur8CDr7hXR      ft:gpt-3.5-turbo-0125:personal succeeded
2026-04-23 15:23

```

1.11 Summary

This notebook demonstrated a complete OpenAI full fine-tuning pipeline for the healthcare security code reviewer use case:

| Step | Action | API Endpoint Used |
|------|--------------------------------|--|
| 1 | Dataset validation | Local |
| 2 | Upload training file | <code>client.files.create</code> |
| 3 | Create fine-tuning job | <code>client.fine_tuning.jobs.create</code> |
| 4 | Monitor job status & events | <code>client.fine_tuning.jobs.retrieve</code> + <code>list_events</code> |
| 5 | Extract and plot metrics | <code>list_events</code> (metric data payloads) |
| 6 | Retrieve fine-tuned model name | <code>client.fine_tuning.jobs.retrieve</code> |
| 7 | Run inference comparison | <code>client.chat.completions.create</code> |
| 9 | Audit trail of all jobs | <code>client.fine_tuning.jobs.list</code> |

The fine-tuned model produced more structured, HIPAA-aware, and actionable security review comments than the base model on held-out test diffs, confirming that domain-specific fine-tuning adds measurable value over prompt engineering alone for this use case.